Sub-Two-Bit Ternary Weight Storage with Table-Free Arithmetic Decoding for CPU Inference

Justus Theile
Independent researcher
justus.theile@gmail.com
https://github.com/purpleskulll/bitnet-t5b

September 2026

Abstract

Ternary (1.58-bit) large language models are stored in practice at two bits per weight, 26 % above the information content of a ternary symbol. We show that the residual redundancy is recoverable on commodity x86 CPUs without lookup tables and without decompressing weights into a wider representation. We introduce **t5b**, a radix-3 code that places five ternary values in one byte at exactly 1.600 bits per weight, and an AVX2 kernel that recovers all five digits from the packed byte using four *independent* `vpmulhuw` instructions per 16-bit view of the byte — eight per 32-byte block — exploiting the fact that a five-trit byte is bounded by 242 and therefore lies well inside the exactness range of the corresponding magic multipliers. The kernel contracts directly on the digits with `vpmaddubsw`, so no weight is ever materialised. We give a roofline argument identifying the condition under which a denser format is profitable — the ratio between a core's multiply-port throughput and its share of memory bandwidth — and verify it by measuring both quantities directly. Integrated into `llama.cpp` as a distinct `ggml` type alongside the existing two-bit format, t5b reduces the shipped `BitNet-b1.58-2B-4T` checkpoint from 1.10 GiB to 1.00 GiB and yields, under `llama-bench` *at four threads*, 1.132× prompt and 1.143× generation throughput, faster in five of five invocations. The re-encoding is lossless and the output is bit-identical: perplexity over 20,480 tokens of WikiText-2 agrees with the two-bit baseline to six significant figures. We further report that 4.76 % of the model's feed-forward neurons are identically zero, that the dead index sets of the gate and up projections coincide in all thirty layers, and that an independent fine-tune revives none of them. All results are from a single AVX2 microarchitecture *without* VNNI. Two independent runs bound that limitation: a static pipeline model, calibrated against the host to 3.0 %, finds the arithmetic ratio stable across Zen 3/4/5, Ice Lake and Sapphire Rapids, and a reviewer's Intel Xeon reproduces it at 2.11 against our 2.43. On real VNNI hardware the same reviewer measures the ratio rising by 8.8 % — the direction predicted, from the predicted mechanism, at a magnitude smaller than the eight-bit storage choice of a system built for those targets would suggest, though on a shared host whose spread exceeds the effect.

1 Introduction

Quantisation to ternary weights $w \in \{-1, 0, +1\}$ preserves task performance at the two-billion-parameter scale while removing multiplication from the dominant matrix operation. The resulting models are attractive for CPU deployment: the matrix–vector product requires only sign application and integer accumulation, and the weight matrix is small enough to change which hardware resource binds the computation.

That last point is usually made as a footprint argument and left there. We take it as the design constraint. During autoregressive generation every weight is read exactly once per emitted

token and used exactly once, so arithmetic intensity is fixed at one multiply–accumulate per stored weight and the achievable token time is bounded below by

$$t_{\text{token}} \;\geq\; \max\!\left(\frac{N\mu}{\pi},\; \frac{N\beta}{8B}\right), \tag{1}$$

where N is the number of weights, β the bits used to store each, μ the instructions issued on the binding execution port per multiply–accumulate, π that port's throughput, and B the memory bandwidth available to the executing core. The first term is arithmetic; the second is transport.

Existing ternary CPU formats occupy the extremes of the (β, μ) trade-off that (1) describes. `llama.cpp`'s `i2_s` uses $\beta = 2$ with a decode of three shifts and four masks per 128 weights. `bitnet.cpp`'s `TL2` and the T-MAC system reach $\beta \approx 1.667$ by table lookup, paying memory for the tables (`TL1` is at 2.0); `llama.cpp`'s own `TQ1_0` reaches 1.6875 by base-3 packing without a table, and `TQ2_0` 2.0625; Spectra 1.1 [7] publishes that family with a correctness theorem and CPU kernels as `TQ1` and `TQ2`, recommending exactly the $p = 8$, $k = 5$ this work uses. Every format named so far is lossless. In the lossy direction, *Sherry* [8] reaches 1.25 bits by imposing 3:4 fine-grained sparsity on the ternary weights and packing four of them into five bits, restoring power-of-two alignment at the cost of a training-time constraint and of exactness — a different trade from the one studied here, where the weights are unchanged bit for bit. In the opposite direction, *Litespark* stores ternary weights at $\beta = 8$, on the explicit grounds that "using 2-bit packing would require unpacking weights before every computation, negating the performance benefit" — a decision made for targets whose dot-product instruction (`VPDPBUSD`, ARM `SDOT`) consumes eight-bit operands.

The base-3 packing itself is not new. `llama.cpp` has carried `TQ1_0` since August 2024 (pull request #8151): its block structure in `ggml-common.h` is annotated *"5 elements per byte (*$3^5 = 243 < 256$*)"*, it decodes without a table, and it has an AVX2 dot product. `TQ1_0` reaches 1.6875 bits per weight only because each 256-element block carries an f16 scale and a 16-element remainder at two bits; the code bytes themselves are at 1.600. Any claim of novelty for the packing would survive on the arithmetic of that block overhead alone, which is not a claim worth making. We had integrated into this very codebase without checking it, and record that here rather than in a footnote.

What this paper contributes is therefore narrower: a per-tensor scale model that removes the block overhead and is drop-in for the `i2_s` path of `bitnet.cpp`, an interleave identical to `i2_s`'s so the access pattern is unchanged, a decode of four *independent* magic multiplies, a column-blocked matrix–matrix kernel, an explicit and measurable condition for when any of this pays, and an end-to-end evaluation on a real ternary checkpoint against the format that ships with it.

What this is and is not. The mechanism is a *lossless re-encoding* of weights that are already ternary, together with a decode-fused kernel. No weight value changes; the output is bit-identical. It is therefore neither pruning nor weight sharing nor a new quantisation scheme, and it composes with all three.

1.1 Contributions

1. A 1.600-bit variant of the *known* base-3 packing (§3). Five ternary values as the base-3 digits of a byte is not ours: it is `TQ1_0`'s construction in `llama.cpp`, and it is published with a correctness theorem by Vaidhya et al. [7] as Spectra 1.1's `TQ1`, whose $p = 8$, $k = 5$ recommendation and 1.6-bit figure are the same as ours. Ours is the scale model — one per-tensor scale in place of `TQ1_0`'s per-256-block f16 and its 16-element two-bit remainder, removing the block overhead entirely (1.600 against 1.6875, 20 % denser than `i2_s`) — with an interleave identical to `i2_s`'s.

2. A decode of depth one rather than depth five (§4). That a multiplication-based decode avoids division and modulo is also known: Spectra 1.1 exploits $3^5 \approx 2^8$ to extract the trits *iteratively*, $b_{i+1} = (3b_i) \wedge \texttt{0xFF}$ for $i = 0 \ldots 4$, a strictly serial chain of five dependent multiplies. Ours is the observation that a five-trit byte satisfies $x \leq 242$, so all four quotients $\lfloor x/3^j \rfloor$ lie inside the exactness range of 16-bit magic multipliers and can be taken **independently, straight from the original byte**, at one `vpmulhuw` each — collapsing the depth-five dependence to depth one. This costs more multiply-port operations than the serial form and buys latency; §5 gives the condition under which that is the right trade. No head-to-head measurement against Spectra's kernel was made: their figures are from a Mac M4 and an EPYC part, ours from one Zen 2 host.

3. An explicit profitability condition (§5) derived from (1) and verified by measuring both π and B rather than inferring either.

4. Integration and end-to-end evaluation (§7) in `llama.cpp`, with the two-bit path retained as a control and losslessness verified by perplexity agreement over 2.05×10^4 tokens.

5. A structural observation (§8): 4.76 % of the checkpoint's feed-forward neurons are identically zero, the dead index sets of W_{gate} and W_{up} coincide in all thirty layers, and an independent fine-tune revives none of them.

2 Background

2.1 The information bound

A ternary weight carries at most

$$H_{\max} = \log_2 3 = 1.58496 \text{ bits}, \tag{2}$$

the origin of the "1.58-bit" label, attained only for a uniform distribution. The empirical marginal over all 2,084,044,800 ternary weights of the shipped checkpoint is

$$(p_{-1}, p_0, p_{+1}) = (0.288940,\ 0.422109,\ 0.288951), \tag{3}$$

giving a zeroth-order entropy

$$H = -\sum_s p_s \log_2 p_s = 1.5603 \text{ bits}. \tag{4}$$

The excess zero mass *lowers* the floor below $\log_2 3$. Storage at $\beta = 2$ exceeds (4) by 28.2 %; the code of §3 exceeds it by 2.5 %.

2.2 The interface a replacement must preserve

`ggml`'s ternary kernels do not compute $\sum_i w_i a_i$ directly. They compute on the shifted code $c_i = w_i + 1 \in \{0, 1, 2\}$, the unsigned operand `vpmaddubsw` requires, and the caller removes the shift:

$$\sum_{i=1}^{n} w_i a_i = \sum_{i=1}^{n} c_i a_i - \sum_{i=1}^{n} a_i. \tag{5}$$

The correction depends only on the activations and is therefore evaluated once per matrix–vector product rather than once per row, $O(n)$ against $O(mn)$. Any replacement must expose the same quantity $\sum_i c_i a_i$. Identity (5) requires $c_i \leq 2$; a code $c = 3$ would contribute $3a_i$ while denoting $w = 0$. We verify in §7.1 that code 3 occurs zero times across all 2,084,044,800 weights.

3 The t5b code

3.1 Definition

A block occupies 32 bytes and carries $L = 160$ ternary weights. For byte index $j \in \{0, \ldots, 31\}$ and digit index $k \in \{0, \ldots, 4\}$,

$$b_j = \sum_{k=0}^{4} c(w_{32k+j})\, 3^k, \qquad c(w) = w + 1 \in \{0, 1, 2\}. \tag{6}$$

The interleave $32k + j$ is `i2_s`'s own layout extended from four digits to five, so digit plane k addresses 32 *consecutive* activations and the memory access pattern is unchanged from the format replaced. The code is injective since $3^5 = 243 \leq 256$, and

$$b_j \;\leq\; 2 \sum_{k=0}^{4} 3^k = 2 \cdot 121 = 242. \tag{7}$$

Density is

$$\beta = \frac{8 \text{ bits}}{5 \text{ weights}} = 1.600, \tag{8}$$

a factor $F = 2/1.6 = 1.25$ fewer bytes than `i2_s`.

3.2 Optimality among aligned radix-3 codes

Packing k trits into $\lceil k \log_2 3 \rceil$ bits gives $\beta(k) = \lceil k \log_2 3 \rceil / k$. Restricting to codes that fit a machine word without straddling:

k	bits	β	fits
4	7	1.750	byte, 1 bit wasted
5	8	**1.600**	byte, exactly
9	15	1.667	16-bit word, 1 bit wasted
10	16	**1.600**	16-bit word, exactly

$k = 5$ and $k = 10$ tie at 1.600 and nothing byte- or word-aligned does better; $k = 10$ is admissible because $3^{10} = 59{,}049 \leq 65{,}536$. We implement both; §4.3 shows they differ by 1.5× in throughput for reasons of SIMD lane width alone, which is why the byte variant is the one deployed.

Relation to Spectra 1.1's Theorem 1. Vaidhya et al. [7] state the *correctness* condition for this family: packing k trits into a p-bit integer is lossless if and only if $2^p > 3^k$, and they likewise recommend $p = 8$, $k = 5$ for an effective 1.6 bits. The table above is the *optimality* counterpart — which admissible (p, k) minimises p/k subject to p being a whole machine word — and every entry in it satisfies their condition by construction. We claim no priority over the condition, the recommendation or the resulting bit rate: all three are theirs, and $k = 5$ is `TQ1_0`'s construction in `llama.cpp` before either. What this subsection adds is the alignment argument making $k = 10$ the only other candidate at the same density, which §4.3 then rules out on lane width.

3.3 Tail

For n not a multiple of L the final block's unused slots take $c = 1$ ($w = 0$) and the corresponding activations are read from a zero-padded buffer, so both terms of (5) are unaffected. Storage is

$$S(n) = 32 \left\lceil \frac{n}{160} \right\rceil \text{ bytes}, \tag{9}$$

exactly 1.600 bits/weight at $K = 2{,}560$ and 1.6296 at $K = 6{,}912$; over the whole checkpoint 1.6076 including tail waste and the 32-byte per-tensor scale record `ggml` appends.

4 Kernel

4.1 Digit recovery without a chain

With $q_j = \lfloor b/3^j \rfloor$ the digits satisfy

$$d_j = q_j - 3q_{j+1}, \qquad j = 0, \ldots, 4, \quad q_5 = 0. \tag{10}$$

A naive implementation derives q_{j+1} from q_j, a serial dependence of length five. We obtain every q_j directly from b, which (7) makes possible.

Proposition 1 (sufficient range for exact magic division)**.** *Let $D \geq 2$ and let M be any positive integer with $e := MD - 2^{16} \geq 1$. Then*

$$\left\lfloor \frac{xM}{2^{16}} \right\rfloor = \left\lfloor \frac{x}{D} \right\rfloor \qquad \textit{for every integer } 0 \leq x < \frac{2^{16} - e(D-1)}{e}. \tag{11}$$

Proof. Write $x = qD + r$ with $0 \leq r < D$. Since $MD = 2^{16} + e$,

$$xM = q\left(2^{16} + e\right) + rM = q\,2^{16} + (qe + rM),$$

so $\lfloor xM/2^{16} \rfloor = q + \lfloor (qe + rM)/2^{16} \rfloor$ and it suffices that $qe + rM < 2^{16}$. Using $q \leq x/D$ and $rM \leq (D-1)M = 2^{16} + e - M$,

$$qe + rM \;\leq\; \frac{xe}{D} + 2^{16} + e - M,$$

which is below 2^{16} exactly when $xe/D < M - e$. Substituting $M = (2^{16} + e)/D$ gives $xe < 2^{16} - e(D-1)$. □

The bound is sufficient, not tight. Since the operative question is finite — does the identity hold for every $x \leq 242$? — we also determine the exact first-failure point by exhaustive search over the whole 16-bit domain, and it is the exhaustive figure the implementation and its tests pin:

j	$D = 3^j$	M	e	bound (11)	first failure	margin over 242
1	3	21,846	2	$x < 32{,}766$	$x = 32{,}768$	135.4×
2	9	7,282	2	$x < 32{,}760$	$x = 32{,}768$	135.4×
3	27	2,428	20	$x < 3{,}251$	$x = 3{,}293$	13.6×
4	81	811	155	$x < 343$	$x = 485$	2.00×

The tightest case first fails at $x = 485$, exactly twice the largest legal packed byte. All four quotients are therefore exact for every input the code can produce, each costs one `vpmulhuw`, and — crucially — the four are mutually independent, so the depth-five chain collapses to depth one and the multiply port rather than latency becomes the limit. Note $M = 811$ is

not the canonical $\lceil 2^{16}/81 \rceil = 810$; the implementation uses the larger constant, which has the smaller exactness range (first failure at 485 rather than 806) and is still twice what the code can produce. The proposition covers it because its hypothesis is only $MD > 2^{16}$. The suite verifies all four identities exhaustively and reports each first-failure point, so the margin is a recorded quantity rather than an assumption. Digits then follow from (10) using shifts and subtractions ($3q = (q \ll 1) + q$) on the unconstrained integer ports.

4.2 Contraction and the accumulator bound

Digit plane k is a register of 32 bytes in $\{0, 1, 2\}$, exactly the unsigned operand `vpmaddubsw` expects. That range is a consequence of (7) and not a property the decode enforces: the thirteen byte values $243 \ldots 255$, which no packer of this format emits, all yield $d_4 = 3$, and a block of them would bound a lane at $2 \cdot 128 \cdot 11 = 2{,}816$, which the twelve-block fold derived below does not survive. What the subtraction needs is weaker and does hold unconditionally — no digit subtraction borrows for any of the 256 byte values.

$$\mathrm{acc}_{16} \mathrel{+}= \texttt{vpmaddubsw}(d_k,\, a_{32k..32k+31})\,. \tag{12}$$

A product is at most $2 \times 128 = 256$ — the bound the header and the code use, covering $a = -128$ — and `vpmaddubsw` sums adjacent pairs, so a 16-bit lane grows by at most 512 per plane and $5 \times 512 = 2560$ per block. Folding to 32-bit every Φ blocks is safe iff

$$2560\,\Phi \le 32{,}767 \iff \Phi \le 12.80, \tag{13}$$

hence $\Phi = 12$. This is a *worst-case* bound. `llama.cpp`'s own `i2_s` kernel folds every 32 blocks with four planes, up to $128 \times 508 = 65{,}024$, and is correct only because signed activations cancel; §7.1 shows that removing the cancellation makes it disagree with exact arithmetic in 200 of 200 cases.

4.3 Why bytes and not words

The ten-trits-per-`uint16` code admits a kernel that never forms a digit. Substituting (10) into $\sum_k d_k a_k$ and re-indexing,

$$\sum_{k=0}^{K-1} d_k a_k = q_0 b_0 + \sum_{j=1}^{K-1} q_j b_j, \qquad b_0 = a_0, \quad b_j = a_j - 3a_{j-1}, \tag{14}$$

so only the quotient chain and a transformed activation vector are needed, the latter computed once per matrix–vector product. The identity holds modulo 2^{32} rather than term by term: intermediate products are large and cancel.

It is nonetheless slower by 1.5×, for reasons of lane width rather than radix. On `uint16` lanes the contraction must use `vpmaddwd`, covering 16 lanes where `vpmaddubsw` covers 32, and the quotient chain competes for the same port. Per 160 weights the word variant issues 19 multiply-port instructions against the byte variant's 13, measuring 30.04 against 33.74 GMAC/s per core (§7).

4.4 Blocking for batched operation

For n_c activation columns, decoding per column costs n_c decodes. Blocking so a decoded plane is contracted against N_C columns held in registers reduces the per-block cost from $N_C(\delta + 5)$ to $\delta + 5N_C$ multiply-port instructions, $\delta = 8$. At $N_C = 8$ this is 48 instructions per 1,280 multiply–accumulates, 26.7 MACs each against `i2_s`'s 32. Measured at $N_C \in \{2, 4, 8\}$: 50.51, 63.54, 73.26 GMAC/s. We deploy $N_C = 8$; its register pressure (181 stack references per loop) costs less than the amortisation gains.

5 When a denser code pays

Let baseline A and candidate B store the same weights at $\beta_A > \beta_B$ with $\mu_A < \mu_B$ binding-port instructions per multiply–accumulate. Write

$$F = \frac{\beta_A}{\beta_B}, \qquad S = \frac{\mu_B}{\mu_A}, \tag{15}$$

and define the *surplus* of the baseline,

$$\Sigma = \frac{\beta_A/(8B)}{\mu_A/\pi} = \frac{\pi\beta_A}{8B\mu_A}. \tag{16}$$

If $\Sigma \le 1$ the baseline is arithmetic-bound and no $S > 1$ can win. If $\Sigma > 1$ it is transport-bound and, from (1), B is faster iff

$$\boxed{S < \Sigma} \tag{17}$$

in which case it runs a factor $\min(F, \Sigma/S)$ faster. The condition is a property of two hardware rates, not of the code, and both are measurable.

5.1 Measuring π

Eight independent dependence chains, so latency cannot bind, with the clock derived from a dependent add chain retiring one per cycle by construction:

instruction	Gop/s	ops/cycle
`vpmaddubsw`	4.09	1.05
`vpmaddwd`	4.10	1.05
`vpmulhuw`	4.09	1.05
`vpmullw`	4.08	1.04
`vpand`	15.43	3.95
`vpaddw`	11.72	3.00

Measured clock 3.91 GHz. All four multiply-class instructions land on one port at 1.05 per cycle; the rest are three to four times more plentiful. This fixes $\pi = 4.09 \times 10^9\,\mathrm{s}^{-1}$ and identifies the binding resource unambiguously.

Methodological note. An earlier version of this measurement gave all eight chains identical seeds; the compiler proved them equal and kept one, eliminating `vpand` entirely and strength-reducing `vpaddw` into a multiply. It reported 2.98×10^6 and 9.15 ops/cycle against a ceiling near 4. Only the physical impossibility of those figures exposed the defect.

5.2 Measuring B: the surplus is a function of core count

B is a core's *share*, not the socket's. Running the baseline kernel at 256 KiB per thread (inside L2) and 64 MiB per thread (four times one CCX's L3), one thread pinned per physical core:

threads	arithmetic (GMAC/s)	transport (GB/s)	Σ
1	83.70	15.47	1.35
2	162.42	24.94	1.63
3	235.14	33.00	1.78
4	287.54	38.62	1.86
6	471.74	38.86	**3.03**

Arithmetic scales 5.6× across six cores, per core falling only 6 %. Transport scales 2.5× and is flat from four cores to six. The surplus therefore grows with core count for reasons independent of any kernel. For t5b, $S = 2.43$ measured against $F = 1.25$; by (17) the code should lose at one, two and four threads and win at six, which is what §7.3 observes, at 0.53×, 0.62×, 0.78× and 1.35× respectively.

6 Experimental setup

CPU	AMD Ryzen 5 3600 (Zen 2), 6 cores / 12 threads
`/proc/cpuinfo`	`avx2`, `fma`; **no AVX-512, no VNNI**
Caches	L1d 32 KiB/core, L2 512 KiB/core, L3 32 MiB in two CCX slices
Memory	62 GiB
Compiler	gcc 13.3.0, `-O3 -mavx2 -mfma -march=native`
Model	`microsoft/bitnet-b1.58-2B-4T`, 2,084,044,800 ternary weights, 210 tensors
Second model	`jpacifico/Aramis-2B-BitNet-b1.58-i2s-GGUF`, independent fine-tune
Harnesses	`llama-bench`, `llama-batched-bench`, `llama-perplexity`

The host is shared with an unrelated production workload whose load average rarely falls below 1.8; within-arm spread ranges from 2 % to 146 % with load, comparable to the effect under study. Every comparison is paired with the arm order alternated, and headline figures are replicated across five independent invocations of the upstream benchmark.

7 Results

7.1 Correctness

Unit level. 184,228 assertions across five suites, zero failures; of these the 177,734 covering the format and both kernel variants are in the public repository and run from a clone with a compiler alone (`make test`). The other three suites cover parts of the wider project not published here. Kernels are checked against scalar references derived independently from the specification rather than from the implemented identities, so a shared derivation error cannot pass. Coverage includes exhaustive verification of the four magic-division identities over their whole legal range, deliberate 32-bit wraparound driven 4.96 times past 2^{31}, extreme activations ±127, and every block and tail boundary.

Model level, argmax. Nine prompts across two models, greedy decoding, one seed: identical output by `diff`. A negative control confirms the comparison can fail.

Model level, distribution. Perplexity over 40 chunks of 512 tokens of WikiText-2, taken from `llama.cpp`'s CI copy:

$$\mathrm{PPL}_{\texttt{i2\_s}} = 96.8243 \pm 3.55673, \qquad \mathrm{PPL}_{\texttt{t5b}} = 96.8243 \pm 3.55673.$$

Not only the final estimates but all forty running estimates agree exactly, from [1] 47.2744 to [40] 96.8243; instrumentation confirms the arms differed (51,660 calls through the new path against zero). This compares a floating-point aggregate over 2.05×10^4 forward passes rather than a decoded string.

How much that assertion count is worth. An assertion count measures effort, not power, so the suite's power is measured directly by mutation: twenty-one defects are injected one at a time into the kernel and its header — each magic multiplier perturbed, the digit multiplication changed from $3y$ to $5y$, the odd-byte view shifted by seven instead of eight, the low-byte mask narrowed, two digit planes given each other's activations, the radix changed, the digit count reduced, the accumulator fold raised from 12 to 13 and to upstream's 32, and six aimed at the column-blocked GEMM path specifically, which shares no code with the per-row one and carries the `pp512` result. **Eighteen are killed.**

One of the six is worth its own sentence. The proposition of §4 gives a *sufficient* exactness range for a magic multiplier, and it is conservative: replacing 811 by 812 is guaranteed only below $x = 198$ but in fact first fails at 323, above every packed byte, so it is not a defect at all. Replacing it by 813 first fails at $x = 242$ — exactly the largest legal packed byte — so it is a defect visible on one input value in 243. The suite kills it.

The remaining three survive, and checking rather than filing them is what makes the result meaningful: all three are *exactly equivalent* on every reachable input, proved exhaustively rather than argued. Replacing the magic multiplier 811 by 810 leaves $\lfloor x/81 \rfloor$ unchanged on all 256 byte values; replacing it by 812 leaves it unchanged for the same reason; and replacing the byte-wise digit subtraction by a word-wise one changes nothing on any of the 65,536 word lanes, because no low byte ever borrows — the invariant §4.3 asserts, confirmed independently of the text that asserts it. The script carries both proofs and fails if either mutant is ever *killed*, since that would mean the kernel had lost the property the proof rests on.

Code 3. Absent: 0 of 2,084,044,800 weights across all 210 tensors, so (5) holds model-wide.

A defect in one of the two baselines. With activations uniformly at ± 127, `llama.cpp`'s `i2_s` *vec_dot* kernel — the reference of §7.1, not the `llamafile_sgemm` path a real run dispatches — disagrees with exact arithmetic on 11,998 **of** 12,000 rows at $K = 6912$, every discrepancy a multiple of 2^{16} — the overflow (13) forbids by construction in our kernel. The prediction that overflow needs $n_b = K/128 \geq 32$, i.e. $K \geq 4096$, is confirmed on both sides: **0 of** 108,000 rows at $K = 2560$ wrap under the same activations.

The shipping path does not have this defect, and reading it is what established that. `tinyBLAS_I2S_AVX` folds `int16` into `int32` once per 128-weight block, inside its block loop, where the reference kernel accumulates 32 blocks first (`group32_num = nb / 32`). There is therefore no 32-block group to overflow on the path that runs, and the finding above is a statement about the reference kernel alone. An earlier version of this paragraph said "`llama.cpp`'s `i2_s` AVX2 kernel" without distinguishing the two, which reads as the stronger claim.

The two exceptions are the bound speaking, not noise, and they change how the claim must be phrased. The margin is $256 \times 127 = 32{,}512$ against 32,767, or 0.78 %, so whether a row overflows turns on its own code mean and a row slightly below average clears the ceiling. At 12,000 rows that tail is visible, so the correct statement is statistical rather than absolute: direction and magnitude hold exactly, "must" does not.

Whether ordinary activations reach that state is not established; the mechanism is. On random `int8` the baseline is exact on all 42,000 rows tried, which is why the two kernels agree wherever it matters.

7.2 Kernel microbenchmark

One core, weights resident in L2 so transport cannot bind:

kernel	GMAC/s	instr	mul	spills	weights	ceiling	of it
`i2_s` (upstream)	81.93	21	4	0	128	130.9	63 %
t10 (word)	30.04	50.25	19.25	8.25	160	34.6	87 %
t5b (byte)	**33.74**	47	13	0	160	50.5	67 %

The ceiling is $\pi \cdot (\text{weights/mul})$. t5b initially measured 24.36 GMAC/s — slower than t10 despite fewer multiplies — which no end-to-end benchmark could diagnose; disassembly showed 44 stack references in a 425-instruction function. One block per iteration with each plane contracted at the moment it exists gave 47 instructions, zero spills, 33.74 GMAC/s, hence $S = 81.93/33.74 = 2.43$.

7.3 Weight traffic of one token

Replaying one token's matrix operations over the model's real shapes and the full weight footprint of both arms together, 940 MB, of which the 521 MB of `i2_s` codes is what a single arm streams, best of five, arms rotated:

threads	`i2_s`	t10	t5b	t5b/`i2_s`
1	34.57 ms	72.73	64.70	0.534
2	20.29	36.84	33.01	0.615
4	13.19	18.82	16.90	0.780
6	15.15	14.49	**11.18**	**1.354**

The crossing matches (17) with $S = 2.43$ and the Σ of §5. Across all eight paired six-thread observations the median is 1.12×, seven of eight favouring t5b; best against best, noting `i2_s` does not improve past four threads, is 13.00 ms against 11.71 ms.

7.4 End to end in `llama.cpp`

	size	pp512 (t/s)	tg128 (t/s)
`i2_s`	1.10 GiB	119.65 ± 2.27	22.56 ± 1.26
t5b	1.00 GiB	$\mathbf{135.45 \pm 1.21}$	$\mathbf{25.78 \pm 0.87}$
ratio	0.909	**1.132**	**1.143**

Faster in five of five invocations on both measures; per-invocation ratios 1.096, 1.112, 1.119, 1.253, 1.132 (prompt) and 1.099, 1.133, 1.123, 1.242, 1.143 (generation).

The four-thread result contradicted §5's prediction; the contradiction is now resolved, and the prediction was right. From $S = 2.43$ and $\Sigma_4 = 1.86$, (17) predicts a loss at four threads and the isolated replay of §7.3 delivers one (0.780×); the model gains 1.132×. Instrumenting the glue with a cycle counter — `rdtsc` at the invariant TSC rate of 3.599978 GHz, not the core clock, with `llama-bench`'s untimed warmup removed by differencing $r = 2$ against $r = 10$ — gives the matmul time directly for one arm and by difference for the other, since everything else a token does is identical between them:

kernel	in situ	replay	ratio
t5b	16.70 ms/token	16.90 ms	**0.988**
`i2_s`	21.18 ms/token	13.19 ms	**1.606**

The replay predicts *our* kernel to 1.2 % and mispredicts `llama.cpp`'s `i2_s` path by 61 %. The effective in-situ ratio is $S_{\text{eff}} = 2.43/1.606 = 1.51$ against $\Sigma_4 = 1.86$, so $S < \Sigma$ *holds* at four threads and (17) predicts the observed gain rather than contradicting it.

What does not survive is the use of the reference `i2_s` kernel as a stand-in for `llama.cpp`'s real performance: it is 1.6× faster than what the model runs, so every ratio computed against it — the 2.43 included — flatters the baseline. No end-to-end number changes. Matmuls are 43.6 % of generation wall time, not the 30 % estimated from the replay. Why the in-situ path costs 1.6× its reference is not established; the dispatch, the accumulator fold and the per-column post-processing are candidates, and isolating them would mean instrumenting the control arm.

The 1.606 **is weaker than the table makes it look, and the integration now carries the fix.** It is not measured but derived, from two `llama-bench` runs taken separately on a host whose load ranged from 2 to 10 (4.905 s against 5.479 s per repetition), so every fluctuation *between* those runs lands entirely in the `i2_s` matmul figure. The integration therefore probes **both** arms at the same dispatch site in the same expression, so the ratio becomes a quotient of two measured quantities and whatever the dispatch costs is common to both and divides out.

The probe's cost is measured rather than assumed. An `rdtsc` pair alone costs 19 ns and is flat in thread count. The same pair with an atomic add to *one shared counter* — what the original instrumentation used — costs 18.7 ns at one thread and 87.6 ns at six, a factor of 4.7, because every `ggml` worker issues a read-modify-write to the same cache line and they serialise on the coherence protocol. Padding the counter per thread returns it to 19 ns, flat. Against the 94.8 µs a single matmul call takes, even the worst case is 0.09 % per probe and 0.18 % for the entry/exit pair, so no version endangers the measurement — but the shared counter's cost *grew along the thread axis this section compares on*, and a systematic error tracking the independent variable is worth removing at any size.

And it has now been run. The measurement needed a patched `llama.cpp`, which had meant a container this host does not grant; building it natively instead — `cmake` and `gcc`, no daemon, no privileges — removed that obstacle. `insitu_measure.sh` interleaves the two arms one repetition at a time rather than running them as two blocks, so a load excursion reaches both at comparable rates, and forms the ratio within each round.

Nine interleaved rounds at four threads, host load 3–5:

	derived (§7.4, earlier)	measured
`i2_s` matmul, ms/token	21.18	21.67
t5b matmul, ms/token	16.70	16.88
ratio	1.268	1.257 (median)
rounds with `i2_s` slower	—	9 of 9

The derived figures survive. The difference method was the right thing to object to and it gave the right answer: 2.3 % on the `i2_s` time, 1.1 % on t5b, 0.9 % on the ratio, with the direct measurement never once favouring the other arm across nine rounds. What the objection bought is not a corrected number but a number that no longer rests on the assumption that everything outside the matmuls costs the same in both arms.

Where that time goes. Two further probes inside `tinyBLAS_I2S_AVX`, one around the contraction loop and one around the per-column post-processing, separate what the dispatch-site probe cannot. Dispatch is the remainder, measured by difference, because a probe around the call would be the call. Five runs:

part of the `i2_s` matmul	share
contraction loop	94.4 %
post-processing	3.1 %
dispatch (by difference)	2.5 %

All three candidates are now answered. Dispatch and post-processing together are 5.6 %, so neither carries the difference: the 1.257 is a statement about the inner loop's own arithmetic and memory traffic. The third, the accumulator fold, is answered by reading the kernel rather than by a third probe — `tinyBLAS_I2S_AVX` has *no* 32-block fold. It folds `int16` into `int32` once per 128-weight block, inside its block loop. So the fold is not a periodic cost that occasionally interrupts the contraction; it is part of every block, and it is inside the 94.4 %.

That also settles which of the two `i2_s` implementations the overflow of §7.1 belongs to, and sharpens why it happens. The kernel feeds `vpmaddubsw` with raw 2-bit codes against `int8` activations, so an `int16` lane takes at most $2 \cdot 3 \cdot 127 = 762$ per instruction and $4 \cdot 762 = 3048$ per block: **ten blocks may be accumulated, not thirty-two**. The reference kernel's 32 is three times its own bound, which is the overflow; the dispatched kernel's 1 is a tenth of it.

The nine omitted folds cost 7.5 % of the dispatched path's matmul time. A tree identical but for accumulating ten blocks before folding measures 9.49×10^9 cycles against 10.25×10^9, three runs each, with *identical* perplexity — 108.6810 on four chunks of WikiText-2 in both. That is one measured component of the 25 % by which the dispatched path trails t5b, not an explanation of it, and it is not a proposal: the `i2_s` path is the control arm of every comparison here, the modified kernel lives in its own tree, and no reported ratio is built from it.

These probes are compiled out by default, and that is not tidiness. They fire once per $RM \times RN$ tile rather than once per call, and they instrument one arm only, so leaving them in inflates the `i2_s` side of the ratio — measured at 1.379 against 1.257 on the same host, a 10 % artefact of the instrument. The decomposition and the ratio come from two builds by design.

One thing this still does not settle: it is one host, the same shared machine as everything else in §7.

Prompt length. 1.106, 1.152, 1.180, 1.113 at 64, 256, 1024 and 2048 tokens: the advantage does not decay with length. We had predicted decay on the grounds that a prompt pass is arithmetic-bound; at these shapes the weights still stream from memory, so the byte saving remains live and the prediction was wrong in its premise.

Concurrency. Total throughput at 1, 2, 4, 8 parallel sequences: 1.041, 1.041, 1.027, 1.097. The advantage *grows*: N concurrent sequences turn generation into a matrix–matrix product with N columns, across which the decode amortises.

Across checkpoints. On the independent fine-tune: identical output on three prompts, 1.120× prompt and 1.075× generation.

Across depth. Repeating the block stack yields a 4.50 B-parameter, 60-layer graph `llama.cpp` loads and runs with real attention and KV cache: 1.165× and 1.129×, both margins clear of their error bars. That artefact is *not a model* — the same thirty blocks run twice — and no quality claim is made from it.

7.5 Two ratios a reader will ask about immediately

Why 25 % fewer weight bytes give 14 % more throughput. At 21.9 tokens per second a token takes 45.7 ms, of which the weight matmuls are 43.6 % — 19.9 ms. That figure is measured in situ by the cycle counter of §7.4, not inferred: the remaining 56.4 % is attention, the KV

cache, RoPE, the norms, the activation quantisation, thread barriers and graph overhead, all identical between the arms, so a 25 % cut in weight bytes acts on under half the token and 14 % is the expected order.

The standalone replay of §7.3 puts the same matmuls at about 14 ms, or 30 %. That gap is the replay's, not the model's: it drives `bitnet_vec_dot_i2_i8_s_reference`, which §7.4 measures at 1.6× the speed of the path `llama.cpp` actually dispatches. The replay is a lower bound on what the matmuls cost and was read as an estimate of it. Read as bandwidth, the generation loop uses 11.4 GB s^{-1} of the 38.9 this machine delivers: it is not at the memory wall either way.

Why the file shrinks by only 9 %. The ternary tensors are 521.0 MB of a 1,187.8 MB file. One f16 tensor — the 128k-vocabulary embedding — is 656.7 MB, 55.3 % of the file, and does not change. The 102.2 MB saved is 19.6 % of what the format touches and 8.6 % of the file. The ternary share, and therefore the file saving, rises with model depth at fixed vocabulary.

7.6 A negative result on row tiling

Tiling four weight rows against one activation vector measures 1.19× in cache and 0.52× from memory. A tile of R rows by C columns loads each weight once and spends it on RC contractions; a matrix–vector product has $C = 1$, so tiling rows buys no reuse and merely reorders loads. Upstream draws the same line structurally: its `ggml_gemv_i2_i8_s` does not tile, while the 4×4 tile lives in `ggml_gemm_i2_i8_s`.

7.7 Against `TQ1_0`, the nearest prior art

§1.1 records that the base-3 packing is `TQ1_0`'s, and that Spectra 1.1 [7] publishes the same $p = 8$, $k = 5$ construction with a correctness theorem. This subsection measures against `TQ1_0`, which is the form of it that ships in the codebase integrated into; no run against Spectra's own kernels was made. The comparison file is produced by re-encoding the same ternary values into `TQ1_0`'s block layout, with the per-tensor `i2_s` scale written into every block's f16 field, so the two files carry identical weights.

	file	ternary payload	achieved bits/weight
`i2_s`	1,187,801,280	521,011,200	2.00000
`TQ1_0`	1,106,386,560	439,603,200	1.68750
t5b	1,085,565,120	418,775,040	1.60755

`TQ1_0`'s 1.6875 is 54 bytes per 256 weights: 48 for the five-per-byte codes, 4 for a 16-element remainder at two bits, and 2 for the f16 block scale. t5b's 1.60755 is 32 bytes per 160 weights plus the tail waste at $K = 6912$ and one 32-byte record per tensor.

On size the honest margin is small. 4.7 % of the ternary payload and 1.9 % of the file. Four fifths of the saving this work reports against `i2_s` was already present in the vendored tree under an upstream type number; the remaining fifth is what the per-tensor scale model contributes.

On speed the margin is large and mostly not about the packing. Three rotated `llama-bench` invocations, all three files in each:

	pp512 (t/s)	tg128 (t/s)
i2_s	122.25 ± 1.52, 77.06 ± 4.49, 118.12 ± 3.26	23.61, 15.27, 23.56
TQ1_0	48.27 ± 0.73, 43.95 ± 5.26, 49.01 ± 0.29	21.68, 16.07, 22.27
t5b	130.65 ± 4.35, 137.38 ± 0.52, 128.17 ± 8.07	22.95, 26.43, 26.43

TQ1_0 loses prompt processing by a factor of about three, and the reason is structural rather than arithmetic: i2_s has a llamafile_sgemm case upstream, t5b has one because we wrote it, and TQ1_0 has none — so it processes a prompt one activation column at a time. **A TQ1_0 sgemm case, which nobody has written, would be expected to erase that margin.** At tg128, where no format has a batched path to exploit, all three land within about 10 % and the ordering does not survive this host's noise.

What survives. Against TQ1_0 on this machine, t5b is worth 4.7 % of the ternary bytes, plus the fact of being wired into the batched path. The claim that a sub-two-bit ternary packing is itself novel does not survive and is withdrawn in §1.1. The narrower claims — the scale model that removes the block overhead, the i2_s-compatible interleave, the independent-quotient decode, the column-blocked kernel and the profitability condition — do.

8 Dead feed-forward neurons

2.9036 % of all weight-matrix rows are identically zero — an output neuron that cannot fire for any input.

tensor	dead rows	share
W_{gate}	9,870 / 207,360	4.76 %
W_{up}	9,870 / 207,360	4.76 %
W_v	103 / 19,200	0.54 %
W_q	4 / 76,800	0.01 %
$W_{\text{down}}, W_k, W_o$	0	0.00 %

They concentrate at the front: 43.59 % of layer 1's feed-forward neurons, 27.58 % of layer 2's, 14.77 % of layer 3's, near zero by layer 8. **In all thirty layers the dead index set of W_{gate} equals that of W_{up}** — not the same cardinality, the same indices. In a SwiGLU block, $h = W_{\text{down}}(\sigma(W_{\text{gate}}x) \odot W_{\text{up}}x)$, the two are combined element-wise, so a zero gate row renders the matching up row irrelevant and conversely; training removed them in pairs. W_{down} has none, the same fact along the other axis: there a dead neuron is a dead *column*.

They survive fine-tuning exactly: against the independent fine-tune, 210 of 210 tensors have identical dead index sets, all 19,847 rows, 100.00 %. Whatever removed them occurred in pre-training.

Skipping the rows in the kernel saves 2.4380 % of weight bytes *and* the same share of multiply–accumulates, and is unconditionally exact. Removing the neurons outright was also attempted, and the outcome is a negative result worth stating because three prior claims about it — two of them ours — were wrong.

It is expressible: llama.cpp reads LLM_KV_FEED_FORWARD_LENGTH through get_key_or_arr into a per-layer array (llama-model.cpp:1117), so the container and the hyper-parameters carry per-layer widths. It is *not* free of loader changes, which an earlier draft of this section asserted without checking: src/models/bitnet.cpp allocates its four feed-forward tensors with the global n_ff, so a file whose layers differ in width fails at load. Four lines change that, and with a scalar feed_forward_length the broadcast makes them a no-op for every existing checkpoint.

With those four lines the pruned model loads — 2.35 B parameters against 2.41 B, 1.08 GiB against 1.10, 60,948,480 ternary weights and 15,268,736 bytes removed. Every surviving weight is bit-identical and every attention tensor byte-identical; the feed-forward block's int8 input is unchanged value for value across all 1,642,496 of them. But it is **not** exact at the token level, and the reason is the sub-layer RMS norm BitNet places between the element-wise product and W_{down}. That norm divides by the root mean over all 6,912 entries, so deleting d of them rescales the output by

$$\sqrt{\frac{K-d}{K}},$$

which in layer 1, where $d/K = 43.59\,\%$, is 0.751 and not a rounding matter. Pinning the divisor to the original width restores it to 2.5 ulp of float32 — not to zero. On three prompts the output is identical on one and diverges on two, late, inside repetitive passages where successive candidates are near-ties and the last bits decide. And it buys nothing measurable: `pp512` $119.24 \pm 3.11 \to 109.90 \pm 9.48$, `tg128` $23.49 \pm 0.51 \to 23.53 \pm 0.72$.

The honest summary is that 2.9 % of the ternary payload can be removed, that doing so costs four lines and changes the output, and that it makes the model no faster. We do not ship it.

9 Limitations

The port analysis is Zen 2 specific — but S is not. §5 measures one multiply-class port at 1.05 instructions per cycle against 3.0 to 4.0 for the cheap classes. A reviewer ran the same benchmark on an Intel part with AVX-512 and obtained 1.60 against 3.09: two 256-bit multiplier ports rather than one. The ratio (17) turns on is therefore a property of the microarchitecture at a finer grain than the instruction set, and the surplus table would have to be re-measured on any other part before (17) could be applied to it.

The worry this raises is sharper than the port count alone, and it deserves its strongest form. Zen 2 cracks every 256-bit integer vector operation into two 128-bit micro-operations issued over four floating-point pipes, retiring roughly two vector operations per cycle; Zen 3 and later, and Ice Lake and later, execute 256 bits natively. The packed kernel spends $42/160 = 0.2625$ vector operations per weight against the baseline's $17/128 = 0.1328$: it is the kernel that issues *more* of them. A machine doubling the vector-op ceiling therefore relieves the packed code preferentially, S could collapse, and the whole motivation for a roofline condition would be an artefact of one narrow part. It does not collapse: §9.1 puts S between 2.10 and 2.45 on Zen 3/4/5, Ice Lake and Sapphire Rapids against 2.50 on Zen 2, and §9.3 measures 2.11 on an Intel part.

One microarchitecture, and the decisive property is absent from it. All results are from a single Zen 2 part reporting `avx2` and `fma` and nothing else. `VPDPBUSD` collapses the contraction into one instruction while leaving the decode unchanged; by (16) this raises S and lowers Σ, moving (17) against a packed code. The *Litespark* system, targeting VNNI and ARM `SDOT`, stores ternary weights at eight bits for precisely this reason. §9.1 *bounds* that move at 0 to 14 % from a static model; §9.2 *measures* it at 8.8 % on hardware we do not own. **The central result is still ours only for AVX2 without VNNI** — every end-to-end number in §7 comes from this one host, and no VNNI machine has run the model. What §9.2 settles is the kernel-level ratio, not the deployed one.

9.1 How far a model can substitute for the hardware

The two objections above differ in one respect that matters: the first concerns code that exists and can be analysed, the second concerns code that does not.

We run `llvm-mca`, LLVM's static pipeline simulator, over the inner loops `gcc -O3 -mavx2 -mfma` actually emits for both kernels, against the vendor-derived scheduler models for six targets. Re-deriving the loop shapes reproduces §7.1's counts exactly for the baseline (21 instructions, 4 multiply-class, 128 weights) and to within one instruction for the packed kernel (46 against 47; the 13 multiplies and 160 weights are exact), the difference being a compiler version.

The model is calibrated before it is used. Its Zen 2 prediction is $S = 2.501$ against the $S = 81.93/33.74 = 2.428$ measured on this host: an error of 3.0 %. The script exits non-zero should that drift past 8 %. Absolute cycle counts are uniformly optimistic by 6 to 11 % (18.01 predicted against 19.7 measured per 160-weight block), which is why only the ratio is carried forward.

target	packed, cyc/block	baseline, cyc/block	S
`znver2` (this host)	18.01	5.76	2.501
`znver3`	11.51	3.76	2.449
`znver4`	11.51	3.76	2.449
`znver5`	11.51	3.76	2.449
`icelake-server`	14.02	5.34	2.098
`sapphirerapids`	14.52	5.01	2.317

These are modelled numbers, and the licence for printing them is that the model was tested twice. Once in-sample, against the host it was calibrated on (3.0 %); once *out of sample*, against a machine it had never seen — the reviewer run of §9.3 measured $S = 2.11$ on an Intel part, where this table predicts 2.098 and 2.317 for the two Intel targets. A model that reproduces its calibration set proves nothing; one that predicts a case withheld from it has earned a table. Every figure here is nonetheless labelled modelled, and the sections it feeds say so at each use.

Two readings would overstate this table. `znver3`, `znver4` and `znver5` return *identical* counts, so they are one prediction and not three: it holds four distinct predictions, not six. And every newer target sits at or below the Zen 2 value rather than scattered around it — the ratio improves by up to 16 % and never degrades — so what is supported is one-sided, that S does not run away, and not that S is constant.

For VNNI the same apparatus is far weaker, and we set out why rather than reporting the number alone. No compiler here targets VNNI usefully, so the two VNNI loops are written by hand: in both kernels the contraction ends in a `vpmaddubsw` followed by an accumulating `vpaddw`, and `VPDPBUSD` performs exactly that pair fused, with the same unsigned-times-signed operand convention. The substitution is argued sound — the reduction width changes from `int16` pairs to `int32` quadruples, leaving the horizontal sum invariant, and it makes the accumulator bound of §4.3 unnecessary — but that is an argument, not a test. **These loops have never been executed.** Three caveats bound what the numbers mean.

1. `VPDPBUSD` accumulates in place with latency 10 to 13, so a loop carrying one accumulator per contraction is bound by that latency until unrolled. On `sapphirerapids` this makes the hand-written VNNI *baseline* slower than the AVX2 one it replaces — 7.01 cycles against 5.01 — which is a property of the accumulator count, not of VNNI. The comparison is therefore read at the resource bound, where an adequately unrolled implementation lands.

2. That metric fails the calibration the dependency-bound metric passed: on Zen 2 it gives $S = 1.96$ against the measured 2.428, an error of -19 %, because the real loop is dependency-bound. Both are reported.

3. The negative control did not fire. `llvm-mca` 23.1.0 assembles and times `VPDPBUSD` for `-mcpu=znver2`, a part without the instruction, and continues to under

`-mattr=-avx512vnni,-avxvnni`, assigning it latency 11 and throughput 1.00. The tool cannot distinguish a legal VNNI sequence from an illegal one and so offers no check on this subsection beyond timing. The control is kept in place and reported failing.

Subject to all three, VNNI moves S in the predicted direction — against the packed code, the baseline spending 4 of 17 operations in the contraction `VPDPBUSD` collapses where the packed kernel spends 5 of 42 — and moves it by 0 to 14 %: S reaches 2.08 on `znver4` and 2.51 on `icelake-server` and `sapphirerapids`. Modest, not decisive.

None of this is a measurement. It also cannot become one by recompiling: no compiler contracts `vpmaddubsw` followed by an accumulating `vpaddw` into `VPDPBUSD` as an idiom, so an AVX2 kernel built with `-march=native` on a VNNI part contains **zero `vpdpbusd`** and measures AVX2 on that part. The instruction has to be written.

`bench_vnni.c` now writes the instruction: `i2_s` and t5b in both forms, the VNNI pair contracting into `int32` accumulators, which for t5b also removes the fold of §4.3 entirely. It verifies with `objdump` at run time that its own binary contains the instruction, checks every row against exact `int64` arithmetic before printing a rate, and exits 3 rather than run on a CPU without the feature. A second build models `VPDPBUSD` in AVX2 so the algorithm can be checked where the instruction cannot execute; it passes on all 100 rows for all four kernels and deliberately prints no timings.

9.2 VNNI, measured

A reviewer ran that benchmark on an Intel Xeon reporting `avx512_vnni`: eight invocations, the ISA check passing on each, all four kernels exact against `int64` arithmetic on every row. **This is the paper's largest limitation moving from modelled to measured**, and it moves only part of the way, so both parts are stated.

	AVX2	VNNI	
median S (`i2_s`/t5b)	2.04	2.22	+8.8 %
`i2_s` gain from VNNI	—	—	1.24×
t5b gain from VNNI	—	—	1.15×
runs in which S rose	—	6 of 8	

The direction is the predicted one and the mechanism is the predicted one. §9.1 argued that `VPDPBUSD` must help the baseline more than the packed format, because the baseline spends 4 of its 17 vector operations in the contraction the instruction collapses while the packed kernel spends 5 of 42. The measurement agrees: 1.24× against 1.15×, and S rises accordingly.

The size is modest and the spread is not smaller than the effect. S rose in six runs of eight, a majority and not a clean separation; the host is a shared two-vCPU sandbox, and the reviewer's own caveat — that run-to-run variation there exceeds the effect being measured — is adopted rather than argued away. An 8.8 % shift in S against Σ values ranging from 1.35 to 3.08 does not by itself decide (17) either way at any thread count.

What the static model got right, and where it was pessimistic. §9.1 bounded VNNI's cost at 0 to 14 % and this measurement lands at 8.8 %, inside that band. The model predicted $S(\text{vnni}) = 2.08$ on `znver4` and 2.51 on the two Intel targets; the measured 2.22 sits between them. Given that the model's negative control failed and its VNNI loops had never been executed, agreement to this degree is more than was claimed for it.

What remains open is a clean machine — or a better design, which is what was done instead. Six of eight is a majority because the eight runs timed the four kernels in sequence, so the AVX2 and VNNI arms of each ratio came from different moments on a drifting host. `bench_vnni.c` now interleaves them: 31 rounds, all four kernels measured in rapid alternation within each round with the starting kernel rotated, S formed inside the round, and

the result reported as a median difference with an exact two-sided sign test. A load excursion then reaches both arms of every ratio instead of landing on one block. That converts a between-condition comparison on a noisy host into a paired one, which is the design a shared machine can actually answer; the numbers above predate it and a dedicated part is no longer the only way to sharpen them. The claim this subsection supports is narrow: on real VNNI hardware the packed format's arithmetic disadvantage grows by roughly a tenth, in the direction and for the reason §9.1 predicted, which is a smaller penalty than the eight-bit storage choice of a system built for those targets would suggest.

9.3 A second microarchitecture, measured

A reviewer ran the benchmarks on an Intel Xeon (AVX-512 capable, 2 vCPU cloud sandbox), giving a genuine second data point for everything except VNNI. Their caveats are adopted rather than paraphrased: with two vCPUs and no pinning, only the one- and two-thread rows carry information; the four-thread `bench_token` row and every `bench_threads` row from $T = 3$ are oversubscribed and mean nothing.

	this host (Zen 2)	reviewer (Intel Xeon)
multiply class, ops/cycle	1.05	1.32 to 1.44
cheap class, ops/cycle	3.0 to 4.0	2.97 to 3.04
`i2_s` reference, GMAC/s/core	81.93	49.86
t5b, GMAC/s/core	33.74	23.61
S	2.43	2.11
Σ at 1 thread	1.35	1.65
Σ at 2 threads	1.63	2.23

Two readings, the second load-bearing. First, S does not collapse on a wider machine: 2.11 against 2.43, two runs within 1 % of each other. Second, **this is an out-of-sample test of §9.1's model, and it passes.** That model predicted $S = 2.098$ for `icelake-server` and 2.317 for `sapphirerapids` — bracketing the measured 2.11 — from a calibration performed only against this Zen 2 host. A static pipeline model tuned on one microarchitecture predicted a second it had never seen, to within 1 %.

At two threads the reviewer's machine gives $S = 2.11 < \Sigma = 2.23$, so (17) predicts a marginal win there and none at one thread. That is a prediction and not a result: no end-to-end run was made on that host, and the elevated Σ comes from the VM's low per-core DRAM bandwidth (8.1 GB s^{-1} single-threaded) rather than from the microarchitecture.

One model size and one architecture family. No public ternary checkpoint larger than 2 B exists. The synthetic 60-layer graph doubles weight traffic but not head count, hidden size or vocabulary.

Measurement environment. The host is shared and run-to-run spread is comparable to the effect: across eight invocations at ordinary load, one has the denser code losing. The headline is therefore the five-invocation replication rather than any single run.

Verification scope. The round trip is exact on all 210 tensors and all 2,084,044,800 weights, and so — since this revision — is the dot-product comparison: 126,000 row-and-activation cases across every `i2_s` tensor, on which the packed kernel matches exact `int64` arithmetic without exception. What remains limited is the read-back from disk, covering the 210 plus the embedding rather than all 332 tensors; the converter's own reread covers the dims, type, offset and alignment of all 332.

Two things this does *not* establish, and they are the ones to hold against it. Nothing here runs the converted model: that the weights survive repacking is not evidence that a loader reading them emits the same tokens, and it cannot be until a loader knows type 43. And exhaustiveness over tensors is not exhaustiveness over inputs — 200 rows per tensor under three activation patterns is a large sample of a space that is not finite.

Task benchmarks. None. The format question does not admit one — output is bit-identical — and where this *model* stands against others is a different question this paper does not address. We ran a cross-model perplexity comparison and do not report it as a result: the checkpoint's GGUF requires `tokenizer.ggml.pre` and the EOS id to be supplied through `--override-kv` because its own metadata is wrong, so an absolute perplexity from it is at least as likely to reflect a tokenisation artefact as the model. It does not affect the identity result, where both arms are affected equally.

Engineering. The blocked matrix–matrix kernel spills 181 times per loop at its chosen width; narrower widths spill less and measure slower. The type identifier used (43) is not reserved by upstream, so a stock `llama.cpp` cannot read the file by construction rather than by accident.

10 Conclusion

The gap between the 1.58 bits a ternary weight carries and the 2.00 it is stored in is recoverable on commodity AVX2 hardware without lookup tables. Five ternary values fit a byte at exactly 1.600 bits; all four radix-3 quotients are extractable from that byte with one independent multiply each, because the byte is bounded by 242 and lies deep inside the exactness range of 16-bit magic multipliers; and the contraction proceeds on the digits, so no weight is materialised.

Whether the resulting 25 % reduction in weight traffic is profitable is not a property of the code. It is decided by (17), a comparison between a core's multiply-port throughput and its share of memory bandwidth, both measurable in minutes. On the part measured here the condition holds from four cores upward, and the deployed model is 9 % smaller and 13 % to 14 % faster with bit-identical output.

The same inequality predicts the result will narrow on a part with VNNI, and §9.2 measures that narrowing at 8.8 % — the direction and mechanism the condition names, at a magnitude the eight-bit storage choice of a system built for such parts would not have led one to expect. That is not a caveat appended to a positive result; it is the result, which is that weight density is a hardware-dependent optimisation with a computable decision rule rather than a property of the model. What §9.1 does settle is that the rule's input is not parochial: across four distinct scheduler models spanning two vendors and three generations, S ranges from 2.10 to 2.50 and every newer target sits at or *below* the Zen 2 value it was derived on, by at most 16 %. The ratio never worsens on a wider machine, so the rule is worth applying rather than an accident of the one part that produced it.

Corrections

Eight claims in earlier drafts of this paper were wrong and were corrected before submission — among them the novelty of the packing, two statements about dead-neuron removal, and the assertion that a run on VNNI hardware would settle §9.1. Each is listed with what replaced it and how it was found in `CHANGELOG.md` in the repository below, kept there rather than here so that this document states what holds.

Data and code availability

The format, the kernels, the benchmarks, the converters and every evidence file cited here are public under the Apache License 2.0 at

`https://github.com/purpleskulll/bitnet-t5b`

with the layout below. No model weights are distributed and no upstream source is redistributed verbatim; the integration is applied as a patch to a checkout the user supplies, and the files that reproduce or adapt an upstream contract say so in their own headers and in `NOTICE`.

claim	file
code histogram, code 3 absence, entropy	`results/tensor_structure.txt`, `results/real_weights_check.txt`
instruction-port throughput π	`benchmarks/bench_ports.c`
surplus Σ against core count	`results/thread_headroom.txt`, `benchmarks/bench_threads.c`
kernel rates, instruction and spill counts	`results/weight_density.txt`, `benchmarks/bench_alu.c`
weight traffic of one token	`benchmarks/bench_token.c`
end-to-end throughput	`results/inference_t5b.txt`
perplexity agreement between the formats	`results/perplexity_t5b.txt`
round trip on real weights	`results/real_weights_check.txt`
dead feed-forward neurons	`results/dead_neurons.txt`, `tools/dead_neurons.py`

The format and its kernels are `src/ternary_t5b.{c,h}` with the `ggml`-facing contract in `src/ggml_t5b_glue.{c,h}` and the suite in `src/test_t5b.c`; the word-lane variant of §4.3 is `src/ternary_t10.{c,h}`; the converter is `tools/gguf_to_t5b.py`.

What the public repository does not contain. The measurements that need a model — §7's end-to-end table, the perplexity agreement — additionally require a `llama.cpp` build carrying the type, a converted GGUF and the checkpoint itself. The integration is supplied as a patch against a named upstream commit rather than as copied sources, and the `i2_s` reference kernel used as the baseline is fetched from that commit by a script rather than redistributed. Everything else — the packing, the kernels, the suite, all four benchmarks, the converters and every evidence file — builds and runs from a clone with a compiler and Python alone.

References

[1] S. Ma, H. Wang, L. Ma, L. Wang, W. Wu, P. Dong, L. Zhang, J. Xue, F. Wei. *The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits.* arXiv:2402.17764, 2024.

[2] Microsoft. *BitNet b1.58 2B4T Technical Report.* arXiv:2504.12285, 2025.

[3] J. Wang, H. Zhou, T. Song, S. Cao, Y. Xia, T. Cao, J. Wei, S. Ma, H. Wang, F. Wei. *Bitnet.cpp: Efficient Edge Inference for Ternary LLMs.* arXiv:2502.11880, 2025.

[4] J. Wang, H. Zhou, T. Song, S. Mao, S. Ma, H. Wang, Y. Xia, F. Wei. *1-bit AI Infra: Part 1.1, Fast and Lossless BitNet b1.58 Inference on CPUs.* arXiv:2410.16144, 2024.

[5] J. Wei, S. Cao, T. Cao, L. Ma, L. Wang, Y. Zhang, M. Yang. *T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge.* EuroSys, 2025. arXiv:2407.00088.

[6] compilade. *ggml-quants: ternary packing for TriLMs and BitNet b1.58*. llama.cpp pull request #8151, August 2024.

[7] T. Vaidhya, A. Kaushal, V. Jain, F. Couture-Harpin, P. Shishodia, M. Behbahani, Y. Nevmyvaka, I. Rish. *Spectra 1.1: Scaling Laws and Efficient Inference for Ternary Language Models*. arXiv:2506.23025, 2025.

[8] H. Huang, D. Wu, Q. Hu, G. Yu, J. Yang, J. Zhu, X. Liu, D. Wu. *Sherry: Hardware-Efficient 1.25-Bit Ternary Quantization via Fine-grained Sparsification*. arXiv:2601.07892, 2026.

[9] *Litespark Inference on Consumer CPUs: Custom SIMD Kernels for Ternary Neural Networks*. arXiv:2605.06485.

[10] S. Han, H. Mao, W. J. Dally. *Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding*. ICLR, 2016. arXiv:1510.00149.

[11] T. Zhang et al. *70% Size, 100% Accuracy: Lossless LLM Compression for Efficient GPU Inference via Dynamic-Length Float*. arXiv:2504.11651, 2025.

[12] G. Gerganov et al. *llama.cpp / ggml*.

[13] S. Merity, C. Xiong, J. Bradbury, R. Socher. *Pointer Sentinel Mixture Models*. arXiv:1609.07843, 2016.